

**Образовательная автономная некоммерческая организация
высшего образования**

«МОСКОВСКИЙ ТЕХНОЛОГИЧЕСКИЙ ИНСТИТУТ»

Факультет информационных технологий
Направление подготовки 09.04.03 «**Прикладная информатика**»

Реферат
«Применение систем управления версиями
в проектах по разработке встроенного ПО»
по дисциплине

«Системы управления версиями»

Выполнил студент гр.
ОВМП-103ивс

Осипов Д. С.

Принял _____
(Должность Руководителя)

Дмитриев Д. В.
(ФИО Руководителя)

Москва, 2026 г.

СОДЕРЖАНИЕ

Введение	3
1. Встроенное ПО как объект управления версиями	5
1.1. Особенности разработки встроенного программного обеспечения.....	5
1.2. Требования к управлению конфигурацией и отраслевые стандарты....	6
2. Инструменты и практики применения СУВ во встроенных проектах	9
2.1. Git и модели ветвления для прошивок	9
2.2. Версионирование прошивок и воспроизводимость сборки.....	10
2.3. Управление зависимостями: SDK, HAL и внешние компоненты.....	11
2.4. Бинарные и аппаратные артефакты	13
2.5. Интеграция с CI/CD и аппаратным тестированием	14
2.6. Практика крупных открытых проектов.....	14
2.7. Рекомендации по организации работы.....	15
Заключение	17
Список использованных источников	18

ВВЕДЕНИЕ

Встроенное программное обеспечение (firmware) управляет огромным числом устройств — от бытовой техники и носимой электроники до промышленных контроллеров, автомобильных электронных блоков и медицинских приборов. С развитием Интернета вещей и «интеллектуальных» встраиваемых систем объём такого ПО и число команд, которые его разрабатывают, постоянно растут. Прошивка современного устройства включает операционную систему реального времени, драйверы, сетевые стеки, средства безопасного обновления и прикладную логику, а над одним продуктом одновременно работают программисты, схемотехники, инженеры по тестированию и специалисты по сертификации.

В таких условиях системы управления версиями (СУВ) перестают быть просто «хранилищем кода». Для встроенного ПО особенно важно точно знать, из каких исходных текстов, с какими версиями библиотек и каким компилятором собрана прошивка, установленная в конкретном устройстве, — иначе невозможно воспроизвести ошибку, выпустить исправление для уже проданной партии или пройти сертификацию. Поэтому тема применения СУВ в проектах по разработке встроенного ПО актуальна как для промышленной практики, так и для подготовки специалистов по интеллектуальным встраиваемым системам.

Цель реферата — проанализировать особенности применения систем управления версиями в проектах встроенного ПО и сформулировать рекомендации по организации работы с ними.

Для достижения цели поставлены задачи:

- рассмотреть специфику разработки встроенного ПО и вытекающие из неё требования к управлению версиями и конфигурацией, в том числе требования отраслевых стандартов;
- проанализировать инструменты и практики: модели ветвления, версионирование прошивок, управление зависимостями, работу с бинарными артефактами, интеграцию с CI/CD;

- изучить опыт крупных открытых проектов (ядро Linux, Zephyr RTOS, ESP-IDF);

- сформулировать практические рекомендации для команды, разрабатывающей встроенное ПО.

Объект исследования — процессы разработки встроенного программного обеспечения. Предмет исследования — методы и инструменты управления версиями и конфигурацией в этих процессах. Работа основана на документации инструментов и проектов, учебной литературе по Git и материалах курса «Системы управления версиями».

1. ВСТРОЕННОЕ ПО КАК ОБЪЕКТ УПРАВЛЕНИЯ ВЕРСИЯМИ

1.1. Особенности разработки встроенного программного обеспечения

Встроенное ПО отличается от прикладных и веб-приложений рядом свойств, которые напрямую влияют на работу с СУВ.

Во-первых, программа тесно связана с аппаратной платформой. Один и тот же исходный код обычно собирается для нескольких плат, ревизий печатных плат и семейств микроконтроллеров, отличающихся периферией и объёмом памяти. В репозитории поэтому хранятся не только исходные тексты, но и описания аппаратуры (devicetree, файлы конфигурации Kconfig, карты памяти, скрипты компоновщика), а изменение одного файла конфигурации может повлиять на десятки целевых устройств.

Во-вторых, сборка выполняется кросс-компиляцией: прошивка для ARM Cortex-M или RISC-V собирается на настольном компьютере специальным набором инструментов (toolchain). Результат зависит от версии компилятора, флагов оптимизации и сторонних SDK, поэтому для воспроизводимости необходимо версионировать не только код, но и окружение сборки.

В-третьих, в проект входят значительные объёмы сторонних компонентов: библиотеки уровня аппаратной абстракции (HAL) от производителей микроконтроллеров, стеки протоколов (TCP/IP, Bluetooth LE), криптографические библиотеки, загрузчики. Эти компоненты развиваются независимо, и проект должен фиксировать точно проверенные их версии.

В-четвёртых, жизненный цикл встроенного продукта длинный. Устройство может эксплуатироваться десять и более лет, и всё это время производитель обязан выпускать исправления безопасности для конкретных выпущенных версий прошивки. Отсюда потребность в долгоживущих ветках сопровождения и возможности точно восстановить состояние проекта на момент любого релиза.

В-пятых, ошибки во встроенном ПО дороги: исправить их после выпуска сложно (нужно обновление «по воздуху» или замена устройства), а в автомобильной, авиационной и медицинской технике они могут угрожать жизни людей. Поэтому разработка таких систем регулируется отраслевыми стандартами функциональной безопасности, требующими строгого управления конфигурацией.

1.2. Требования к управлению конфигурацией и отраслевые стандарты

В стандартах функциональной безопасности управление версиями рассматривается как часть более широкого процесса — управления конфигурацией (configuration management). Его задача — однозначно идентифицировать каждый элемент конфигурации (исходный файл, требование, тестовый сценарий, инструмент сборки), контролировать изменения и в любой момент восстанавливать согласованное состояние продукта. В таблице 1 приведены основные стандарты и их требования к этому процессу.

Таблица 1 — Отраслевые стандарты и требования к управлению конфигурацией

Стандарт	Область	Требования к управлению конфигурацией
IEC 61508	Функциональная безопасность электрических и электронных систем (базовый стандарт)	Идентификация элементов конфигурации, контроль изменений и анализ их влияния, сохранение базовых версий
ISO 26262 (часть 8)	Автомобильная электроника	Отдельный поддерживающий процесс управления конфигурацией: уникальная идентификация рабочих продуктов, воспроизводимость, трассируемость изменений
DO-178C	Бортовое ПО авиационной техники	Процесс управления конфигурацией ПО: базовые версии (baselines), отчёты о проблемах, контроль изменений, архивирование и восстановление
IEC 62304	ПО медицинских изделий	Процесс управления конфигурацией ПО: идентификация компонентов, включая сторонние (SOUP), контроль изменений, учёт состава версий

Из требований стандартов вытекают конкретные требования к СУВ и практикам работы с ней:

- идентификация версий — каждая выпущенная прошивка должна соответствовать неизменяемой метке в репозитории (тегу) и уникальному номеру версии внутри самой прошивки;
- трассируемость — для любого изменения должно быть известно, кто, когда и зачем его внёс, с какой задачей и с каким требованием оно связано и какую проверку прошло;
- контроль изменений — изменения в защищённые ветки попадают

только после рецензирования и автоматических проверок;

— воспроизводимость — по метке версии можно заново получить ту же самую прошивку, включая версии всех зависимостей и инструментов;

— сохранность — история и артефакты релизов хранятся с резервным копированием на весь срок жизни продукта.

Распределённые СУВ, и прежде всего Git, хорошо подходят для выполнения этих требований: каждый коммит идентифицируется криптографическим хешем, что делает незаметную подмену содержимого истории практически невозможной, а платформы GitHub и GitLab добавляют защищённые ветки, обязательное рецензирование и журналы аудита. Однако сам по себе Git не гарантирует соблюдения требований — они достигаются за счёт правил работы команды и автоматизации, рассмотренных во второй главе.

2. ИНСТРУМЕНТЫ И ПРАКТИКИ ПРИМЕНЕНИЯ СУВ ВО ВСТРОЕННЫХ ПРОЕКТАХ

2.1. Git и модели ветвления для прошивок

Сегодня Git является фактическим стандартом и во встроенной разработке: на нём ведутся ядро Linux, операционные системы реального времени Zephyr и FreeRTOS, фреймворк ESP-IDF, проекты сборки встраиваемых дистрибутивов Yocto и Buildroot. Централизованные системы (Subversion, Perforce) по-прежнему встречаются в крупных компаниях с унаследованными репозиториями и большими объёмами бинарных данных, но новые проекты, как правило, начинают с Git.

Выбор модели ветвления во встроенных проектах определяется длинным жизненным циклом продукта. На практике обычно сочетаются два подхода:

- основная ветка разработки (main) с короткоживущими ветками задач, которые сливаются через pull request или merge request после рецензирования и прохождения CI — вариант trunk-based development или GitHub Flow;
- долгоживущие ветки релизов и сопровождения (например, v3.7-branch в Zephyr или release/v5.x в ESP-IDF), куда переносятся только исправления ошибок и уязвимостей.

Исправление обычно сначала вносится в основную ветку, а затем переносится (backport) в поддерживаемые ветки релизов командой git cherry-pick или средствами автоматизации. Такой порядок гарантирует, что ошибка, исправленная в старой версии, не вернётся в новой. Для продуктов, которые сертифицируются, релизная ветка дополнительно «замораживается»: любое изменение в ней проходит формальную процедуру анализа влияния.

Полноценная модель Git Flow с ветками develop, release и hotfix применяется реже: она хорошо подходит для продуктов с редкими плановыми выпусками, но при частой интеграции создаёт лишние слияния. Важнее не конкретная модель, а её явное описание в документации проекта и

защита ключевых веток настройками платформы.

2.2. Версионирование прошивок и воспроизводимость сборки

Каждый релиз прошивки отмечается аннотированным тегом в репозитории (например, v2.4.0), а номер версии назначается по правилам семантического версионирования: изменение старшего номера означает несовместимые изменения, среднего — новые возможности, младшего — исправления ошибок. Номер версии должен быть «зашит» в саму прошивку, чтобы его можно было прочитать с работающего устройства, из отчёта об ошибке или в процессе обновления.

Распространённый приём — формировать строку версии при сборке командой `git describe --tags --dirty`. Она выдаёт ближайший тег, число коммитов после него и сокращённый хеш, например `v2.4.0-3-g1a2b3c4-dirty`. По такой строке разработчик однозначно находит коммит, из которого собрана прошивка, а суффикс `dirty` предупреждает, что сборка сделана из рабочей копии с незафиксированными изменениями, — такие сборки не должны попадать к заказчику. Этот механизм используют, в частности, системы сборки Zephyr и ESP-IDF.

Однако одного хеша коммита недостаточно, если результат зависит от окружения. Для воспроизводимости сборки (*reproducible builds*) применяются следующие меры:

- фиксация версии компилятора и SDK — например, сборка в Docker-контейнере, образ которого описан в репозитории, или указание версии `toolchain` в манифесте;
- фиксация версий всех внешних компонентов (подробнее — в разделе 2.3);
- устранение недетерминированности: временные метки в прошивке берутся из переменной `SOURCE_DATE_EPOCH` (обычно — время последнего коммита), а не из текущего времени;
- хранение вместе с релизом контрольных сумм бинарных файлов и файла конфигурации сборки.

В результате по тегу из репозитория можно через годы получить побитово ту же прошивку, что установлена в устройствах, и выпустить для неё минимальное исправление.

2.3. Управление зависимостями: SDK, HAL и внешние компоненты

Встроенный проект почти всегда собирается из нескольких репозиторий: собственного кода, ОС реального времени, HAL-библиотек производителей, сетевых стеков. Git предлагает несколько способов связать их, а экосистемы встроенной разработки создали собственные инструменты (таблица 2).

Таблица 2 — Механизмы управления внешними зависимостями во встроенных проектах

Механизм	Принцип	Применение и
Git submodules	В основном репозитории хранится ссылка на конкретный коммит вложенного репозитория	ESP-IDF и многие проекты на микроконтроллерах. Точная фиксация версий, но легко забыть git submodule update; неудобно при большом числе компонентов
Git subtree	Код зависимости копируется в дерево основного репозитория с сохранением истории	Небольшие библиотеки, которые нужно дорабатывать на месте. Нет отдельной команды для клонирования, но репозиторий растёт
West (манифест west.yml)	Мета-инструмент Zephyr: в манифесте перечислены репозитории и их ревизии	Zephyr RTOS и проекты на его основе (например, nRF Connect SDK). Удобное управление десятками HAL-модулей
Репо (XML-манифест)	Инструмент Google для работы с сотнями Git-репозиториями по манифесту	Android Open Source Project, иногда Yocto-сборки
Слои и рецепты Yocto (SRCREV)	Рецепт BitBake указывает URL репозитория и хеш коммита исходников	Сборка встраиваемых Linux-дистрибутивов; версии всех пакетов фиксируются в слоях
Менеджеры компонентов	Зависимости описываются в манифесте с диапазонами версий и загружаются из реестра	IDF Component Manager (ESP-IDF), PlatformIO, CMSIS-Pack

Общий принцип для всех механизмов: версия каждого внешнего компонента должна быть зафиксирована в репозитории точным коммитом или тегом, а её изменение — проходить как обычное изменение кода, с рецензированием и тестами. Ссылки на «последнюю версию» ветки допустимы только в экспериментальных сборках. Для сертифицируемых продуктов, кроме того, составляется перечень сторонних компонентов (SOUP по IEC 62304, SBOM — software bill of materials) с их версиями и

лицензиями; этот перечень удобно генерировать автоматически из манифестов.

Отдельный вопрос — доработки сторонних компонентов. Если в HAL-библиотеку производителя приходится вносить исправления, их лучше вести в собственном форке на отдельной ветке и по возможности отправлять в исходный проект. Тогда при обновлении библиотеки видно, какие локальные изменения нужно перенести, а не теряются правки, сделанные «прямо в скопированных файлах».

2.4. Бинарные и аппаратные артефакты

Помимо исходного кода, встроенный проект порождает большое количество бинарных данных: собранные образы прошивок, загрузчики, бинарные «блобы» производителей (например, прошивки радиомодулей), файлы проектов ПЛИС, схемы и печатные платы, документацию. Хранение крупных и часто меняющихся бинарных файлов в обычном Git приводит к росту репозитория, поскольку каждый клон получает все их версии.

Для решения этой проблемы применяются следующие подходы:

- Git LFS (Large File Storage) — в репозитории хранятся только небольшие указатели, а содержимое крупных файлов загружается с отдельного сервера по мере необходимости; поддерживаются блокировки файлов (`git lfs lock`) для данных, которые нельзя сливать автоматически;
- репозитории артефактов (JFrog Artifactory, Sonatype Nexus, релизы GitHub/GitLab) — собранные прошивки хранятся там с привязкой к тегу и хешу коммита, а не в самом Git;
- правило «не коммитить результаты сборки» — каталоги `build/`, объектные файлы и образы исключаются через `.gitignore`, а релизный образ всегда можно пересобрать;
- текстовые форматы для аппаратных проектов — современные САПР печатных плат (например, KiCad) хранят схемы в текстовом виде, что позволяет версионировать их в Git и сравнивать изменения.

Если объём бинарных данных очень велик (например, в проектах с графическим интерфейсом и мультимедиа), компании используют Perforce Helix Core или сочетание Git для кода и отдельной системы для ассетов.

2.5. Интеграция с CI/CD и аппаратным тестированием

Репозиторий становится центром автоматизации: каждое событие (push, создание pull request, установка тега) запускает конвейер непрерывной интеграции. Для встроенного ПО он обычно включает следующие этапы:

- статический анализ и проверки стиля (clang-format, checkpatch, MISRA-совместимые анализаторы), проверку формата сообщений коммитов;
- сборку для всех поддерживаемых плат и конфигураций — матрица сборок позволяет сразу увидеть, что изменение «сломало» одну из платформ;
- модульные тесты на хост-машине и тесты в эмуляторах (QEMU, Renode) без реального оборудования;
- тестирование на реальных платах (hardware-in-the-loop, HIL): стенд из нескольких отладочных плат подключён к серверу CI, прошивка автоматически загружается и проверяется;
- при установке тега — сборку релизных образов, их цифровую подпись, публикацию в репозиторий артефактов и подготовку пакета обновления «по воздуху» (OTA).

Результаты проверок отображаются прямо в pull request, а правила защиты ветки не позволяют слить изменение, пока они не пройдены. Тем самым СУВ и CI вместе обеспечивают требование стандартов о контролируемом внесении изменений: в истории остаётся запись о том, кто одобрил изменение и какие проверки оно прошло.

2.6. Практика крупных открытых проектов

Ядро Linux. Именно для разработки ядра Линус Торвальдс в 2005 году создал Git, когда проект лишился возможности бесплатно использовать

проприетарную систему BitKeeper. Процесс разработки ядра иерархический: изменения в виде патчей обсуждаются в почтовых рассылках подсистем, принимаются их мейнтейнерами (ответственные указаны в файле MAINTAINERS) и поступают в основное дерево через их Git-репозитории. Каждый патч содержит строку Signed-off-by — подтверждение автора по Developer Certificate of Origin (DCO), что он вправе передать этот код. Новая версия ядра выходит примерно раз в 9–10 недель, а отдельные версии получают статус долгосрочной поддержки (LTS) и сопровождаются годами — что особенно важно для встраиваемых систем на Linux.

Zephyr RTOS. Проект Linux Foundation для микроконтроллеров использует GitHub и строгие правила: формат сообщения коммита «подсистема: область: описание» проверяется инструментом gitlint, обязательна подпись Signed-off-by, изменения рецензируют ответственные из файла MAINTAINERS.yml. Внешние модули (HAL производителей, криптобиблиотеки) подключаются через мета-инструмент west, а версии с долгосрочной поддержкой ведутся в отдельных ветках vX.Y-branch, куда переносятся исправления. CI-система проекта с помощью инструмента twister собирает и тестирует код на сотнях плат и эмуляторов.

ESP-IDF. Фреймворк Espressif для микроконтроллеров ESP32 использует Git-подмодули для сторонних компонентов, ветки release/vX.Y для поддерживаемых версий и чёткую политику поддержки: для каждой версии объявляется период сопровождения, в течение которого выпускаются исправления. Дополнительно проект предлагает менеджер компонентов, позволяющий подключать библиотеки из реестра с указанием версий.

Опыт этих проектов показывает общие черты: строгая идентификация версий через теги, долгоживущие ветки поддержки с переносом исправлений, явная фиксация версий внешних компонентов, обязательное рецензирование и массовое автоматическое тестирование на разных платформах.

2.7. Рекомендации по организации работы

На основе рассмотренного материала можно сформулировать

рекомендации для команды, разрабатывающей встроенное ПО:

- использовать Git с платформой (GitHub, GitLab или собственным сервером), включить защиту основной и релизных веток, обязательное рецензирование и прохождение CI перед слиянием;
- описать модель ветвления в CONTRIBUTING.md: основная ветка, короткоживущие ветки задач, ветки сопровождения релизов и порядок переноса исправлений;
- отмечать каждый выпуск аннотированным (по возможности подписанным) тегом, встраивать в прошивку строку `git describe` и запрещать передачу заказчику «dirty»-сборок;
- фиксировать версии всех внешних компонентов и инструментов сборки в репозитории (манифест `west`, подмодули, Docker-образ), формировать SBOM;
- не хранить в Git результаты сборки; крупные бинарные файлы вести через Git LFS или репозиторий артефактов;
- принять единый формат сообщений коммитов со ссылкой на задачу и проверять его автоматически — это обеспечивает трассируемость, требуемую стандартами;
- выполнять сборку для всех целевых плат и тестирование в эмуляторах при каждом изменении, а для релизных веток — тестирование на реальном оборудовании;
- регулярно создавать резервные копии репозитория, LFS-хранилища и настроек CI, поскольку продукт придётся сопровождать многие годы.

ЗАКЛЮЧЕНИЕ

В реферате рассмотрено применение систем управления версиями в проектах по разработке встроенного программного обеспечения. Показано, что специфика встроенного ПО — тесная связь с аппаратурой, кросс-компиляция, большое число сторонних компонентов, длинный жизненный цикл и высокая цена ошибки — предъявляет к СУВ повышенные требования. Помимо хранения истории кода, система должна обеспечивать идентификацию каждой выпущенной прошивки, трассируемость изменений, контроль их внесения и воспроизводимость сборки, что прямо требуют стандарты функциональной безопасности IEC 61508, ISO 26262, DO-178C и IEC 62304.

Анализ инструментов показал, что фактическим стандартом стал Git, дополненный практиками и средствами, учитывающими особенности отрасли: долгоживущими ветками сопровождения с переносом исправлений, тегами и встраиванием версии в прошивку через `git describe`, мета-инструментами управления зависимостями (`west`, `perio`, подмодули, рецепты Yocto), Git LFS и репозиториями артефактов для бинарных данных, а также конвейерами CI/CD со сборкой для множества плат, эмуляцией и тестированием на реальном оборудовании. Опыт ядра Linux, Zephyr RTOS и ESP-IDF подтверждает эффективность этих практик в больших распределённых командах.

Сформулированные рекомендации могут быть использованы при организации разработки интеллектуальных встраиваемых систем. Перспективным направлением дальнейшего изучения является автоматизация требований стандартов функциональной безопасности средствами платформ разработки — автоматическое построение матриц трассируемости и SBOM на основе истории репозитория.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — Apress, 2014. — URL: <https://git-scm.com/book/ru/v2> (дата обращения: 05.10.2026).
2. Git Documentation. — URL: <https://git-scm.com/docs> (дата обращения: 05.10.2026).
3. Сергеев О. В. Теоретический материал по курсу «Системы управления версиями». — М.: МТИ, 2025.
4. Дмитриев Д. В. Системы управления версиями: презентации лекций. — М.: МТИ, 2026.
5. The Linux Kernel documentation: Submitting patches: the essential guide to getting your code into the kernel. — URL: <https://docs.kernel.org/process/submitting-patches.html> (дата обращения: 05.10.2026).
6. The Linux Kernel documentation: How the development process works. — URL: <https://docs.kernel.org/process/2.Process.html> (дата обращения: 05.10.2026).
7. Zephyr Project Documentation: West (Zephyr's meta-tool). — URL: <https://docs.zephyrproject.org/latest/develop/west/index.html> (дата обращения: 05.10.2026).
8. Zephyr Project Documentation: Contribution Guidelines. — URL: <https://docs.zephyrproject.org/latest/contribute/guidelines.html> (дата обращения: 05.10.2026).
9. ESP-IDF Programming Guide: ESP-IDF Versions. — URL: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/versions.html> (дата обращения: 05.10.2026).
10. Yocto Project Documentation. — URL: <https://docs.yoctoproject.org> (дата обращения: 05.10.2026).
11. Git Large File Storage. — URL: <https://git-lfs.com> (дата обращения: 05.10.2026).
12. Reproducible Builds: SOURCE_DATE_EPOCH specification. — URL: <https://reproducible-builds.org/specs/source-date-epoch/> (дата обращения: 05.10.2026).

05.10.2026).

13. ISO 26262-8:2018. Road vehicles — Functional safety — Part 8: Supporting processes. — Geneva: ISO, 2018.

14. IEC 62304:2006+A1:2015. Medical device software — Software life cycle processes. — Geneva: IEC, 2015.

15. RTCA DO-178C. Software Considerations in Airborne Systems and Equipment Certification. — Washington: RTCA, 2011.

16. IEC 61508-3:2010. Functional safety of electrical/electronic/programmable electronic safety-related systems — Part 3: Software requirements. — Geneva: IEC, 2010.